

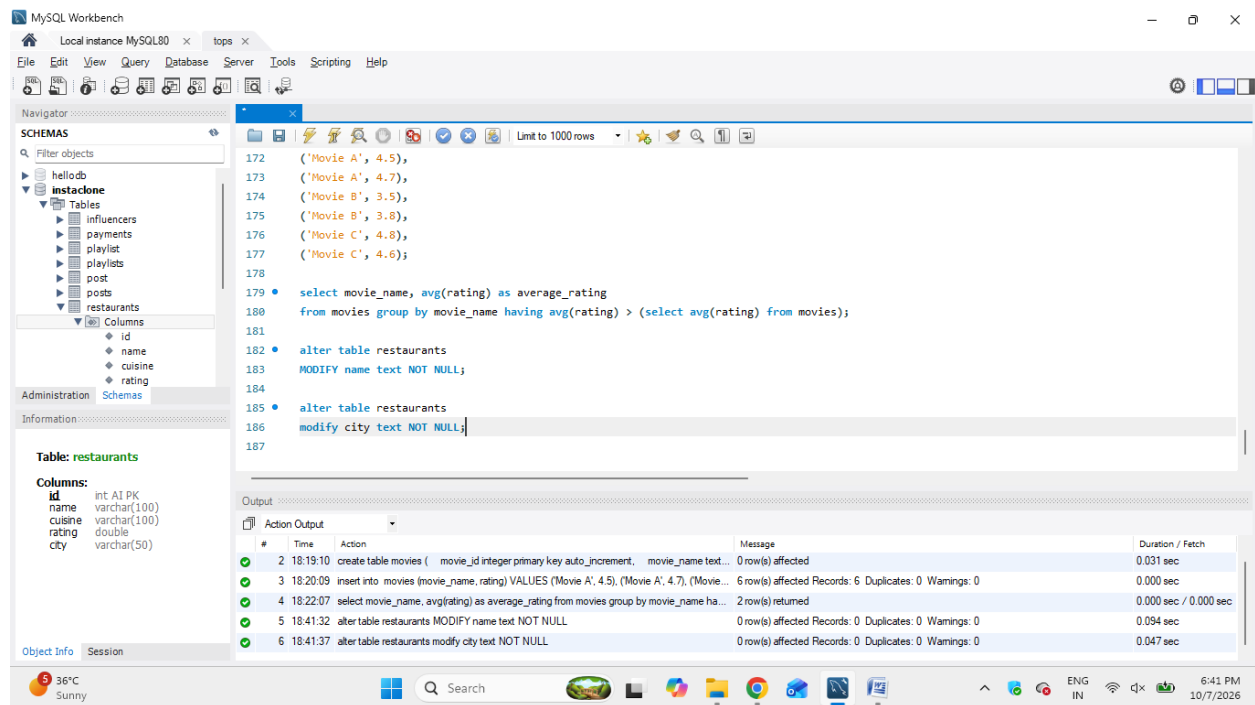
Session 07 Constraints & Views

Trainer Coverage: Constraints: NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY. Creating and querying VIEWS. Example: Create “ActiveUserView” showing users with >5 transactions.

Q:-01 Create a table called Restaurant with columns: id, name, location, and cuisine. Apply NOT NULL constraint to name and location, and make id the PRIMARY KEY.

alter table restaurants modify name text NOT NULL;

alter table restaurants modify city text NOT NULL;



Q:-02 Create a table called FoodOrder with columns: order_id, restaurant_id, user_id, and order_total. Set order_id as PRIMARY KEY, and add a FOREIGN KEY constraint on restaurant_id referencing Restaurant(id).

**create table foodOrder (
order_id integer primary key,**

restaurant_id integer,
user_id integer,
order_total real,
foreign key(restaurant_id) REFERENCES restaurants(id));

Q:-03 Insert 6 rows into a table called SpotifyUser with columns: user_id, username, email, and subscription_type. Ensure that username is UNIQUE and email is NOT NULL.

Hint: Use the UNIQUE and NOT NULL constraints when creating the table.

insert into SpotifyUser

(user_id, username, email, subscription_type) values

- (1, 'Mital', 'mital@gmail.com', 'Premium'),
- (2, 'Nidhi', 'nidhi@gmail.com', 'Free'),
- (3, 'Teja', 'teja@gmail.com', 'Premium'),
- (4, 'Rahul', 'rahul@gmail.com', 'Premium'),
- (5, 'Priya', 'priya@gmail.com', 'Free'),
- (6, 'Amit', 'amit@gmail.com', 'Premium');

The screenshot displays the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows the 'instacore' database selected. The 'Tables' list includes 'influencers', 'payments', 'playlist', 'post', 'posts', and 'restaurants'. The 'Columns' list for 'restaurants' includes 'id', 'name', 'cuisine', 'rating', and 'city'. The 'Table: restaurants' section shows its columns and data types: 'id' (int AI PK), 'name' (varchar(100)), 'cuisine' (varchar(100)), 'rating' (double), and 'city' (varchar(50)).

The main editor shows the following SQL code:

```
204 (1, 'Mital', 'mital@gmail.com', 'Premium'),
205 (2, 'Nidhi', 'nidhi@gmail.com', 'Free'),
206 (3, 'Teja', 'teja@gmail.com', 'Premium'),
207 (4, 'Rahul', 'rahul@gmail.com', 'Premium'),
208 (5, 'Priya', 'priya@gmail.com', 'Free'),
209 (6, 'Amit', 'amit@gmail.com', 'Premium');
210
211 select * from SpotifyUser;
```

The 'Result Grid' shows the output of the 'select * from SpotifyUser;' query, displaying 6 rows of data:

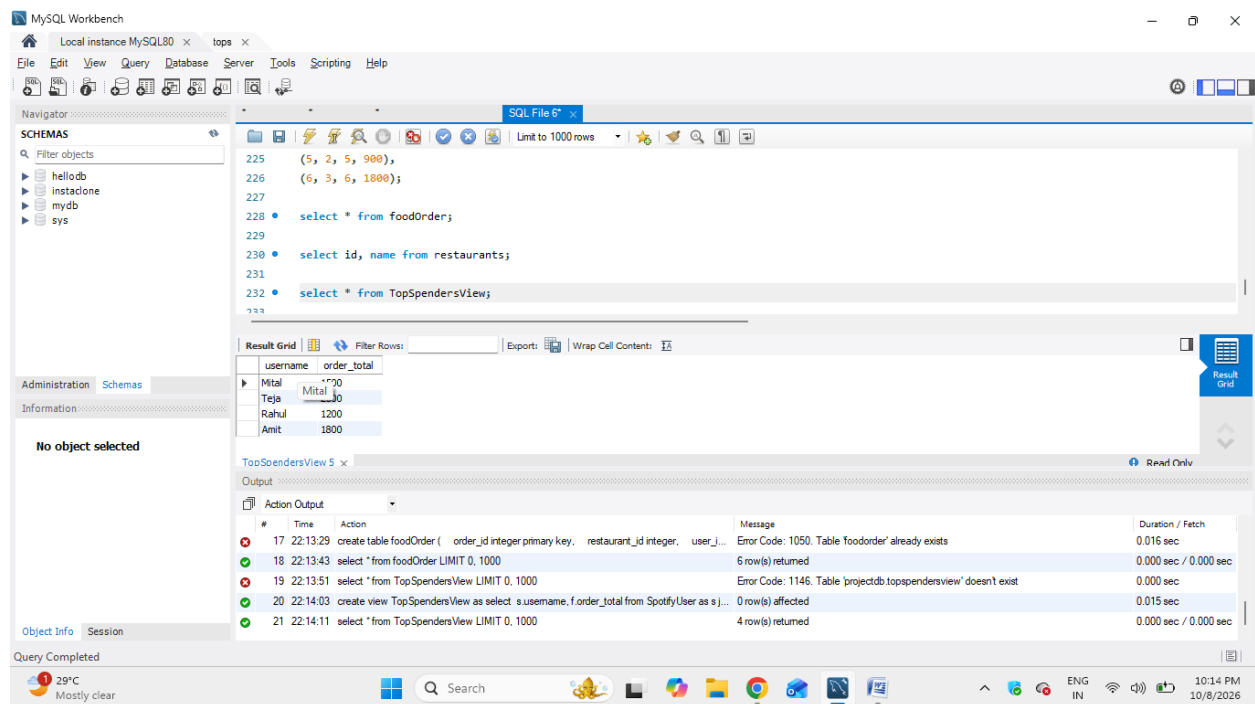
user_id	username	email	subscription_type
1	Mital	mital@gmail.com	Premium
2	Nidhi	nidhi@gmail.com	Free
3	Teja	teja@gmail.com	Premium
4	Rahul	rahul@gmail.com	Premium
5	Priya	priya@gmail.com	Free
6	Amit	amit@gmail.com	Premium

The 'Action Output' pane shows the execution of the following SQL statements:

- 7 18:46:27 create table foodOrder (order_id integer primary key, restaurant_id integer, user_id integer, order_total real, foreign key(restaurant_id) REFERENCES restaurants(id)); 0 row(s) affected 0.047 sec
- 8 18:55:19 create table SpotifyUser (user_id integer primary key, username text unique, email varchar(100) not null, subscription_type enum('Free', 'Premium')) 0 row(s) affected 0.016 sec
- 9 18:56:13 create table SpotifyUser (user_id integer primary key, username varchar(100) unique, email varchar(100) not null, subscription_type enum('Free', 'Premium')) 0 row(s) affected 0.046 sec
- 10 18:56:59 insert into SpotifyUser (user_id, username, email, subscription_type) VALUES (1, 'Mital', 'mital@gmail.com', 'Premium'), (2, 'Nidhi', 'nidhi@gmail.com', 'Free'), (3, 'Teja', 'teja@gmail.com', 'Premium'), (4, 'Rahul', 'rahul@gmail.com', 'Premium'), (5, 'Priya', 'priya@gmail.com', 'Free'), (6, 'Amit', 'amit@gmail.com', 'Premium'); 6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0 0.015 sec
- 11 18:57:21 select * from SpotifyUser LIMIT 0, 1000 6 row(s) returned 0.000 sec / 0.015 sec

Q:-04. Create a VIEW named TopSpendersView that shows usernames and order_total from a FoodOrder table where order_total is greater than 1000.

**create view TopSpendersView as select
s.username, f.order_total
from SpotifyUser as s
join foodOrder as f
on s.user_id = f.user_id
where f.order_total > 1000;**



Q:-05. Write a SQL query to list all users from the TopSpendersView who have spent more than 2000, sorted by order_total descending. **Constraint: Only use the TopSpendersView for your query, not the original FoodOrder table.**

**select username, order_total from TopSpendersView
where order_total > 2000 order by order_total desc;**

MySQL Workbench

Local instance MySQL80 x tops x

File Edit View Query Database Server Tools Scripting Help

Navigator

SQL File 6"

230 • select id, name from restaurants;
231
232 • select * from TopSpendersView;
233
234 • select * from SpotifyUser;
235
236 • select username, order_total from TopSpendersView
237 where order_total > 2000 order by order_total desc;
238

Result Grid

username	order_total
Teja	2500

Administration Schemas

Information

No object selected

Object Info Session

Query Completed

29°C Mostly clear

Search

ENG IN

10:18 PM 10/8/2026

Output

Action Output

#	Time	Action	Message	Duration / Fetch
20	22:14:03	create view TopSpendersView as select s.username, f.order_total from SpotifyUser as s j...	0 row(s) affected	0.015 sec
21	22:14:11	select * from TopSpendersView LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
22	22:17:44	select username, order_total from TopSpendersView where order_total > 2000 order by or...	1 row(s) returned	0.016 sec / 0.000 sec
23	22:17:49	select * from TopSpendersView LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
24	22:17:58	select username, order_total from TopSpendersView where order_total > 2000 order by or...	1 row(s) returned	0.000 sec / 0.000 sec